DS 210 Final Project Report
Stella Louie

Project Overview

Goal

The goal of this project is to find out if a student's habits can help predict how well they do on their exams. I wanted to answer the question: *"*Can daily behaviors like studying, sleeping, and social media use predict academic performance?*"*

Dataset

I used a dataset called `student_habits_performance.csv`, which contains information about each student's habits and their exam score. Each row in the dataset represents one student. It includes things like study hours, attendance, sleep hours, and more. The dataset was created for the project and loaded from a local CSV file.

How?

I used a decision tree to do this which learns from examples of students and their exam scores, and then makes predictions on the exam score range (like low, medium, or high) for a new student based on their habits.

I also used a graph to find students who have similar habits. I measured how alike two students are by comparing their habits using Euclidean distance. If two students are very similar, they get connected in the graph. Then I used breadth-first search (BFS) to explore the graph and find all the students who are similar to a selected student.

In summary, this project helps show how everyday habits might relate to school performance. It also lets us find students who are similar to each other, which could help teachers or students understand good habits for doing well in school.

Data Processing

```
// this function loads students from a CSV file

// inputs: a file path pointing to the CSV file

// output: a list (vector) of Student objects

// high-level logic: it uses a CSV reader to go through each line in the file and
turns each row into a Student and adds it to a list



fn load_students(file_path: &str) -> Result<Vec<Student>, Box<dyn Error>> {
```

```rust
    let mut rdr = ReaderBuilder::new().has_headers(true).from_path(file_path)?;

    let mut students = Vec::new();



    // goes through each row in the CSV and converts it to a Student

    for result in rdr.deserialize() {

        let student: Student = result?;

        students.push(student);

    }



    Ok(students)

}
```

I loaded the data into Rust using the `csv` crate, which makes it easy to read CSV files. Each row from the file is turned into a `Student` struct, which holds all the information about a student. The program automatically converts each line of the CSV into a `Student` using the `serde` crate. I also converted text values (like “Male” or “Yes”) into numbers using helper functions like `encode_gender()` and `encode_bool()`, so the machine learning model could understand them.

Code Structure

Modules

The code is split into 3 main modules:

- `utils.rs`: defines the Student struct and handles converting their habits into numbers.
- `model.rs`: contains the logic for training the decision tree model and making predictions.
- `graph.rs`: builds a similarity graph between students based on how similar their habits are, and runs BFS on that graph.

Key Functions & Types

Student struct (in utils.rs)
Purpose: Stores student data and has methods to turn that data into numbers.

Inputs/Outputs: Uses values like age and gender; outputs a vector of numbers (features).
Logic: Encodes non-number info like gender or internet quality into numerical values.

train_model() (in model.rs)
Purpose: builds a decision tree from the student data
Inputs: a list of Students.
Outputs: a trained model that can make predictions
Logic: turns each student's habits into numbers and uses them to teach the model

predict() (in model.rs)
Purpose: predicts a student's exam score category
Inputs: the trained model and a new Student
Outputs: a number representing their exam score category (0 = low, 1 = medium, 2 = high)
Logic: feeds the student's features into the model.

create_similarity_graph() (in graph.rs)
Purpose: connects students who have similar habits using a graph
Inputs: list of Students
Outputs: a graph and a map to track student IDs
Logic: uses Euclidean distance to measure similarity and connects close students

run_bfs_from_start() (in graph.rs)
Purpose: finds all students similar to a given student by exploring the graph
Inputs: graph and a starting student node
Outputs: list of similar students
Logic: uses BFS to go through connected students

Main Workflow: The main function (`main.rs`) loads student data from the CSV. It trains a model using `train_model()` and stores it. Through the menu, I can enter a new student's data and use `predict()` to get their predicted exam score range. Then I can also enter a student ID, and through creating a graph, the code runs BFS to find similar students.

Testing

```
running 4 tests
test tests::test_euclidean_distance_zero_for_identical ... ok
test tests::test_encode_features_length_and_values ... ok
test tests::test_predict_with_known_student ... ok
test tests::test_load_students_from_sample_csv ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

In this project, I added a few simple tests to make sure the main parts of my code are working correctly.

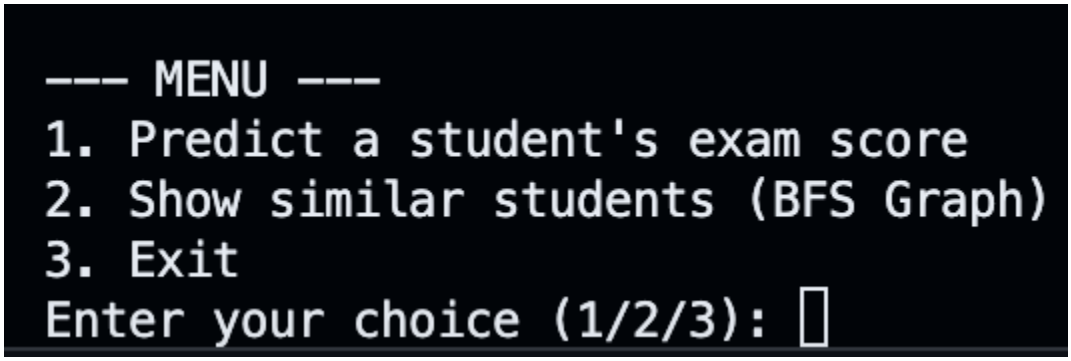
The first test, `test_load_students_from_sample_csv`, checks if the program can read student data from a CSV file properly. This matters because the rest of the program depends on that data being accurate and correct.

The second test, `test_predict_with_known_student`, checks if the prediction model gives the right result for a student we already know. This helps confirm that the prediction system works.

Another test, `test_encode_features_length_and_values`, ensures that the student's information gets turned into the right number of values (features), which is important when using that data in a machine learning model.

The last test, `test_euclidean_distance_zero_for_identical`, checks that when we compare a student to themselves, the similarity score is exactly zero. This confirms that the similarity math is correct.

Results & Usage



When I ran the program, it successfully loaded student data and allowed me to interact with it using a simple text menu. I was able to enter new student habit information and get a prediction for what range their exam score would fall into.

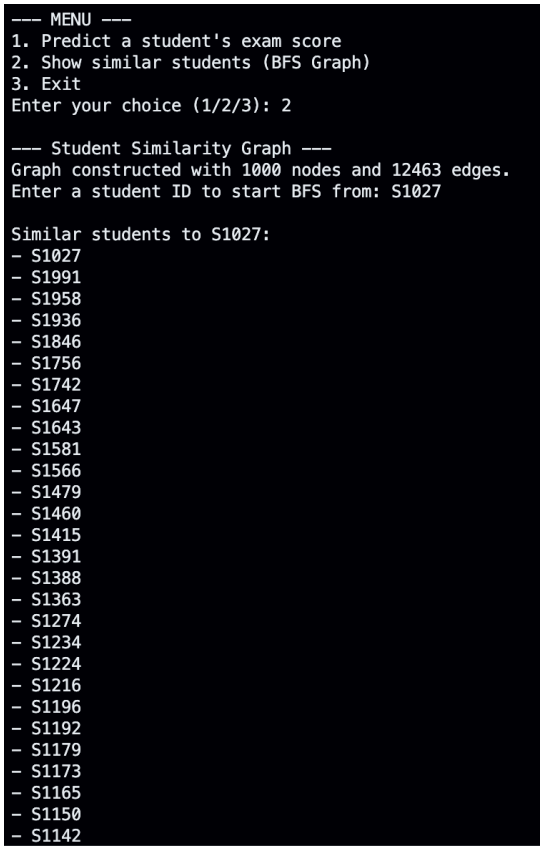
I also tested the feature that finds similar students based on habits. By entering a student ID from the data, the program returned a list of other students who had similar behaviors, using a graph and Breadth-First Search (BFS) to explore connections.

To run this project, open your terminal and go to the project folder. Then, type `cargo run` to build and run the program. The program will load student data from a CSV file and show a menu in the terminal. You can choose options by typing 1, 2, or 3. Option 1 lets you enter details about a new student, and the program will predict their exam score range. Option 2 asks for a student ID and shows other students with similar habits using a graph. Option 3 exits the program.

```
--- MENU ---
1. Predict a student's exam score
2. Show similar students (BFS Graph)
3. Exit
Enter your choice (1/2/3): 1

--- Enter student details ---
Student ID: S1027
Age: 18
Gender (Male/Female): Male
Study hours per day: 3.2
Social media hours per day: 2.2
Netflix hours per day: 2.8
Part-time job? (Yes/No): Yes
Attendance percentage: 88.1
Sleep hours per day: 4.8
Diet quality (Poor/Fair/Good): Fair
Exercise frequency per week: 5
Parental education level (High School/Bachelor/Master): Bachelor
Internet quality (Poor/Average/Good): Average
Mental health rating (1-10): 3
Extracurricular participation? (Yes/No): No
 Predicted exam score category: Between 60 and 79
```

Here is an example of Option 1. We used student S1027 as an example and we got the predicted exam score category of between 60 to 79.



Here is an example when Option 2 is selected in the menu. It shows other students with similar habits using a graph.

Sources

I used https://docs.rs/serde/latest/serde/trait.Deserialize.html to learn more about deserializing and how to implement it in my code. From my understanding, deserializing is the process of turning data from a file or string (like a line from a CSV or JSON file) into a Rust struct or object that you can work with in code. In my project, deserializing occurs when the function reads rows from the CSV file and converts them into `Student` structs. This is done using the `serde` library, which knows how to match the column names in the file to the fields in the struct.

I also used https://docs.rs/linfa/latest/linfa/ to learn more about linfa and the comprehensive toolkit that it provides to help with the DecisionTree. In my project, linfa is the machine learning library used to train a model that can predict a student's exam score category based on their habits. Here, I give it student data (with inputs like age, study hours, sleep, etc. and output exam scores), and it learns patterns from that data using a model (a decision tree). After training, I can give it to a new student, and it will predict their exam score range.

I also used ChatGPT to learn how to use commit in GitHub so that I could document the progress of my code. I also researched gitlog so that I could keep track of how many commits I have/am currently using.